

Hub Técnico - Busk Seguros



Bienvenido al **Hub Técnico de Busk Seguros**. Este portal centraliza toda la documentación técnica para el sistema automatizado de procesamiento de documentos.

Flujo Crítico de Procesamiento

Para entender cómo el sistema maneja los archivos desde el origen hasta la base de datos, consulta:  [Proceso FTP y Validación](#)

Navegación de Especificaciones

- [Arquitectura](#): Componentes del sistema y diagramas de flujo.
- [Firmas de API](#): Endpoints REST y contratos de datos.
- [Base de Datos](#): Modelo ER y diccionario de datos de PostgreSQL.
- [Catálogo de Productos](#): Estructura detallada de los **Anexos 1 al 5**.
- [Lógica de Negocio](#): Reglas de Stock, Inclusiones y Novedades.

[!TIP] Utiliza la barra lateral para navegar o la búsqueda global para encontrar campos específicos de los anexos.

Arquitectura de Infraestructura

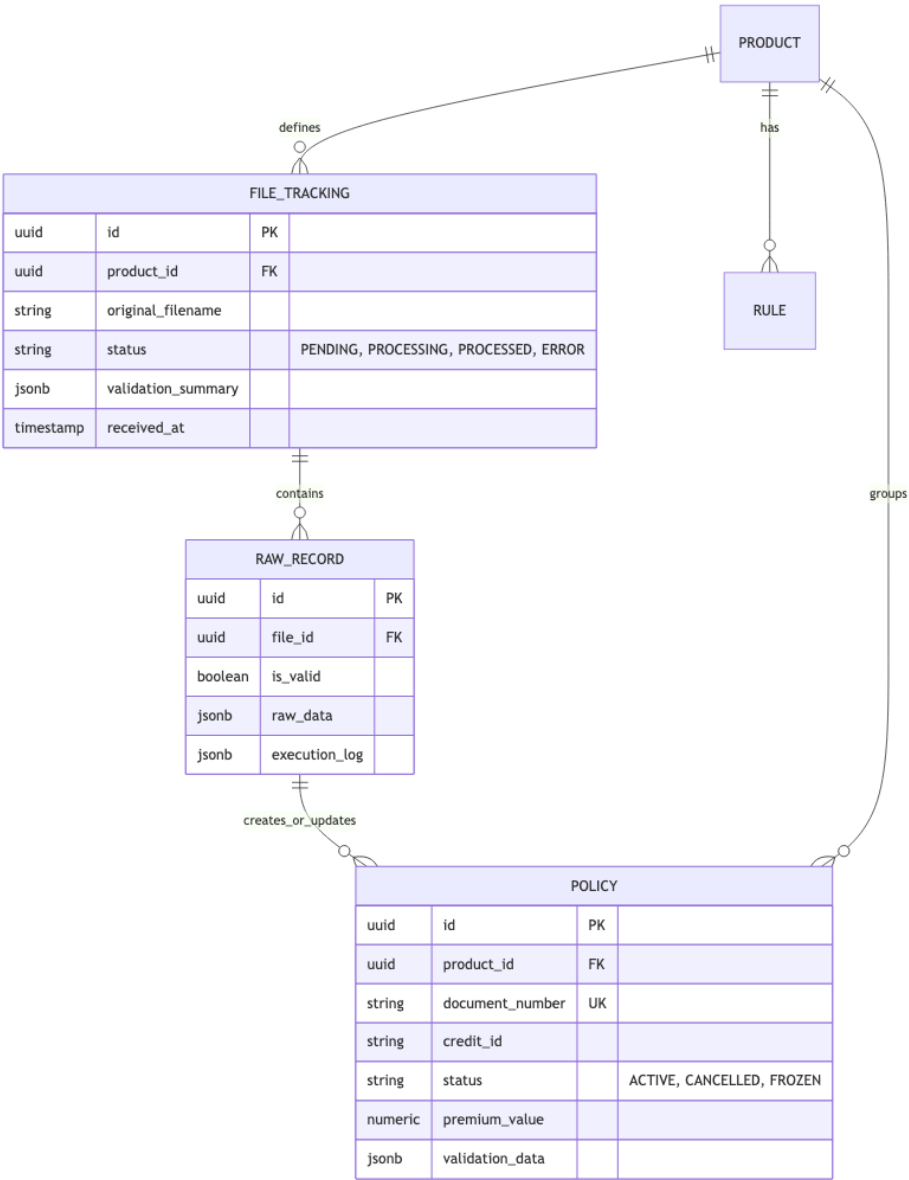
Representación visual de los componentes del sistema y el ciclo de vida del procesamiento de documentos.

1. Modelos Visuales de Referencia

Para comprender rápidamente la magnitud y funcionamiento del sistema, revisa los modelos conceptuales aprobados:

Modelo de Entidades (ERD)

Para claridad conceptual, a continuación las entidades core y sus relaciones principales en la base de datos:



```

erDiagram
    FILE_TRACKING ||--o{ RAW_RECORD : contains
    RAW_RECORD ||--o{ POLICY : creates_or_updates
    PRODUCT ||--o{ FILE_TRACKING : defines
    PRODUCT ||--o{ RULE : has
    PRODUCT ||--o{ POLICY : groups

    FILE_TRACKING {
        uuid id PK
        uuid product_id FK
        string original_filename
        string status "PENDING, PROCESSING, PROCESSED, ERROR"
        jsonb validation_summary
        timestamp received_at
    }

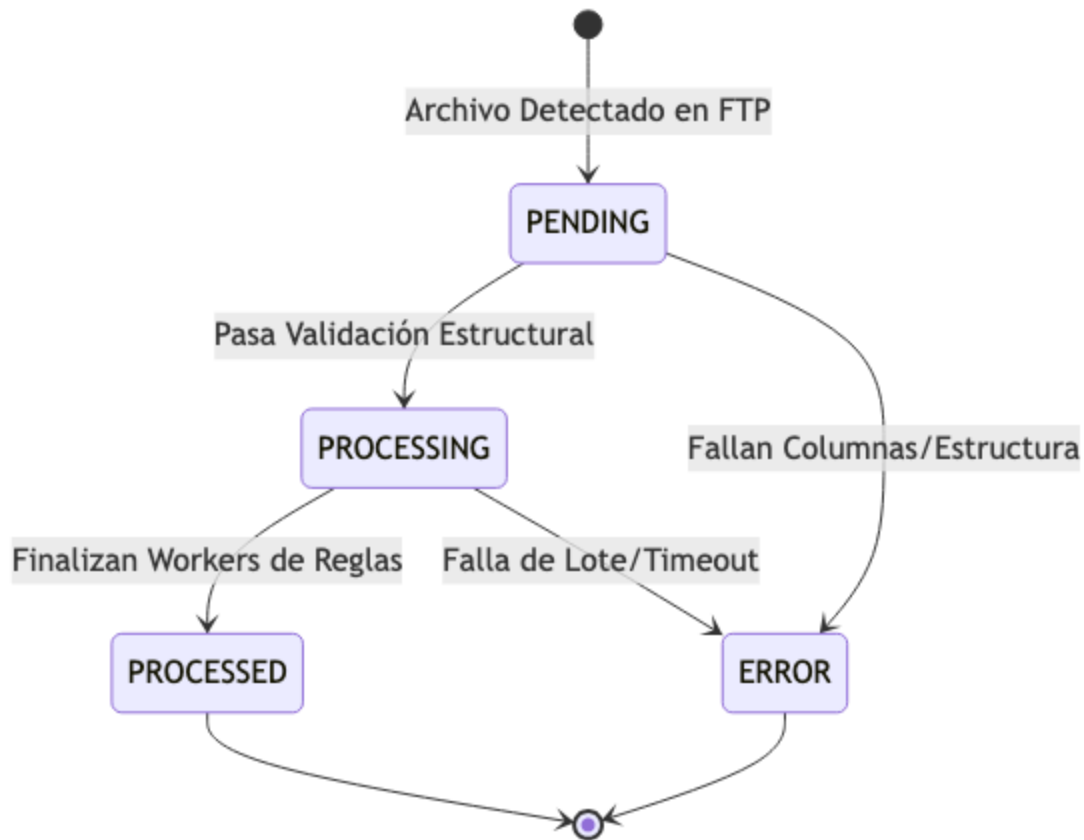
    POLICY {
        uuid id PK
        uuid product_id FK
        string document_number UK
        string credit_id
        string status "ACTIVE, CANCELLED, FROZEN"
        numeric premium_value
        jsonb validation_data
    }

    RAW_RECORD {
        uuid id PK
        uuid file_id FK
        boolean is_valid
        jsonb raw_data
        jsonb execution_log
    }

```

Ciclo de Vida del Archivo (File Tracking)

Controla el estado operativo del procesamiento en bloque de un Anexo FTP.



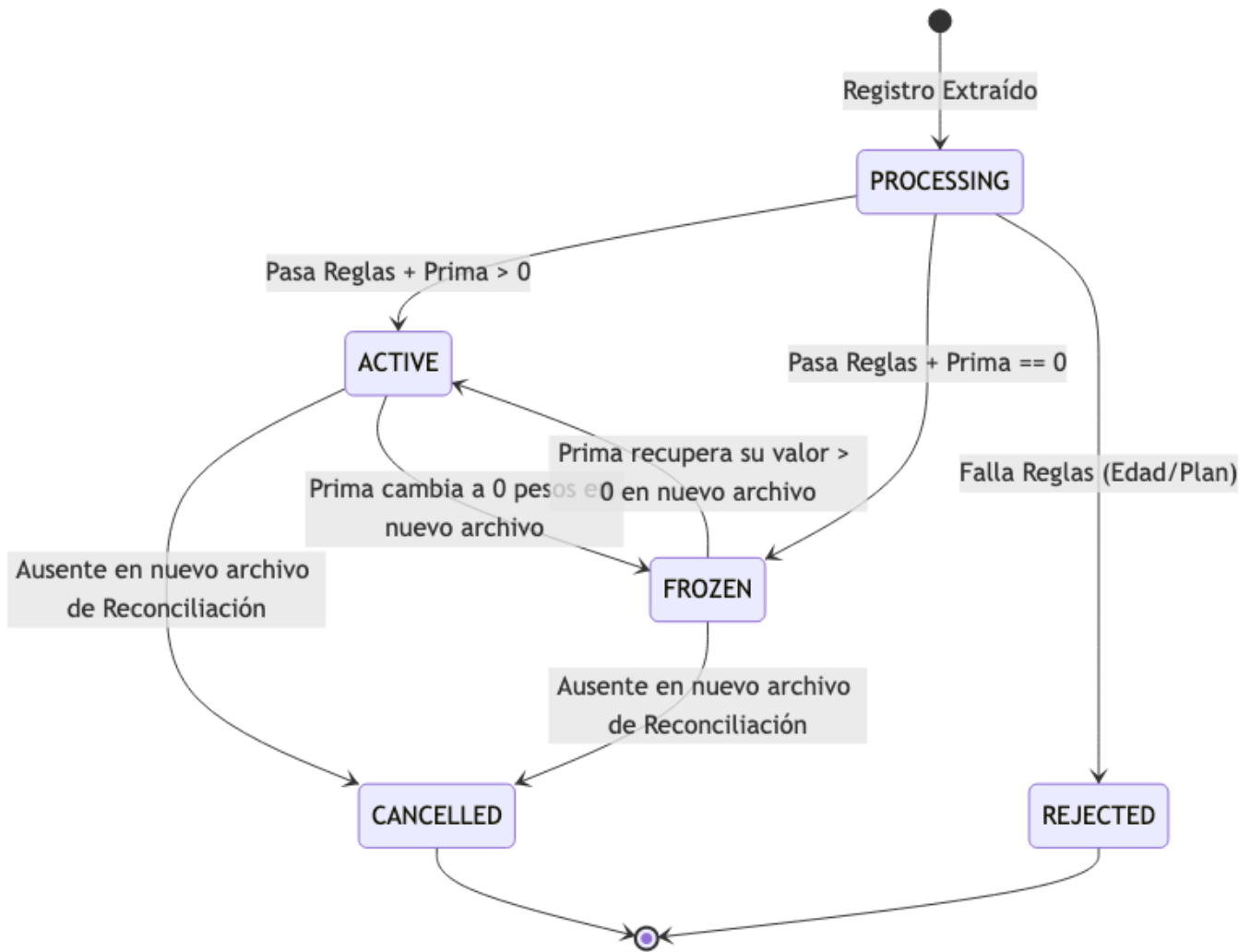
stateDiagram-v2

```

[*] --> PENDING: Archivo Detectado en FTP
PENDING --> PROCESSING: Pasa Validación Estructural
PENDING --> ERROR: Fallan Columnas/Estructura
PROCESSING --> PROCESSED: Finalizan Workers de Reglas
PROCESSING --> ERROR: Falla de Lote/Timeout
PROCESSED --> [*]
ERROR --> [*]
  
```

Flujo de Estados de Póliza (Policy)

A nivel de fila única individual; el ciclo de vida de un asegurado/deudor a partir de las reglas asíncronas.



stateDiagram-v2

[*] --> PROCESSING: Registro Extraído

PROCESSING --> ACTIVE: Pasa Reglas + Prima > 0

PROCESSING --> FROZEN: Pasa Reglas + Prima == 0

PROCESSING --> REJECTED: Falla Reglas (Edad/Plan)

ACTIVE --> CANCELLED: Ausente en nuevo archivo de Reconciliación

FROZEN --> CANCELLED: Ausente en nuevo archivo de Reconciliación

ACTIVE --> FROZEN: Prima cambia a 0 pesos en nuevo archivo

FROZEN --> ACTIVE: Prima recupera su valor > 0 en nuevo archivo

CANCELLED --> [*]

REJECTED --> [*]

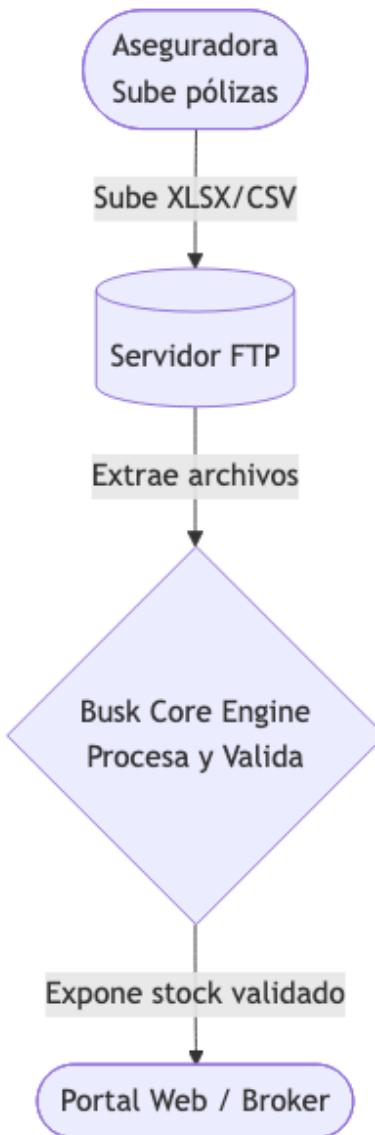
Modelo de Componentes General (Estructura de Carpetas)

- /api: Endpoints REST Fiber/Gin.

- /jobs: Background Workers (FTP, Reconciliación).
 - /rules: Motor de Validación Síncrono y Asíncrono.
 - /models: Structs (Entidades) de la BD.
-

2. Arquitectura de Componentes (C4 Model)

El sistema está diseñado en Go (Golang) para alto rendimiento, interactuando con un servidor FTP externo y una base de datos PostgreSQL.



flowchart TD

A([Aseguradora \n Sube pólizas])

B([Servidor FTP])

C{Busk Core Engine \n Procesa y Valida}

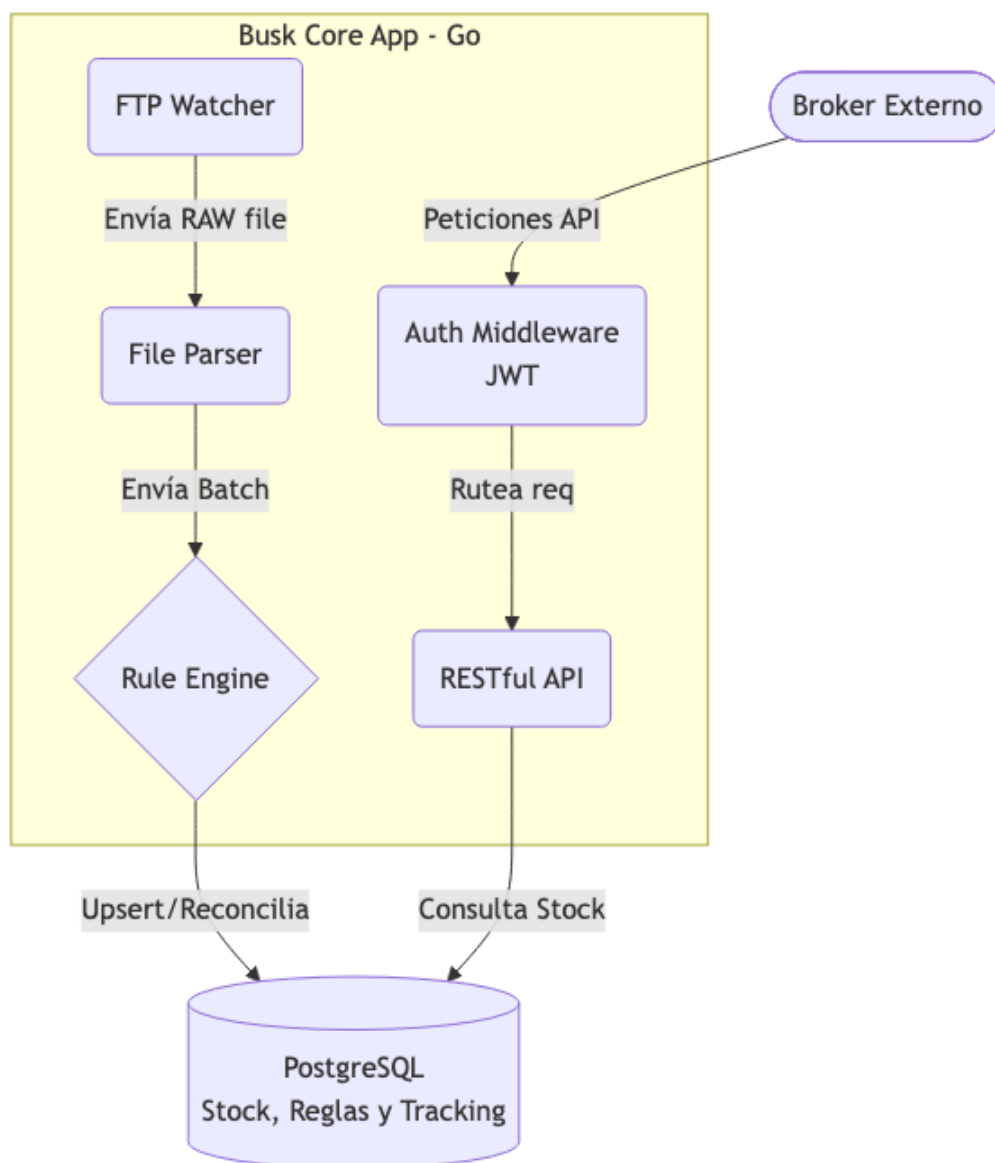
D([Portal Web / Broker])

A -- Sube XLSX/CSV --> B

B -- Extrae archivos --> C

C -- Expone stock validado --> D

Componentes Internos (Go)



```

flowchart TD
    DB[(PostgreSQL \n Stock, Reglas y Tracking)]

    subgraph Core [Busk Core App - Go]
        direction TB
        W(FTP Watcher)
        P(File Parser)
        RE{Rule Engine}
        Auth(Auth Middleware \n JWT)
        API(RESTful API)
    end

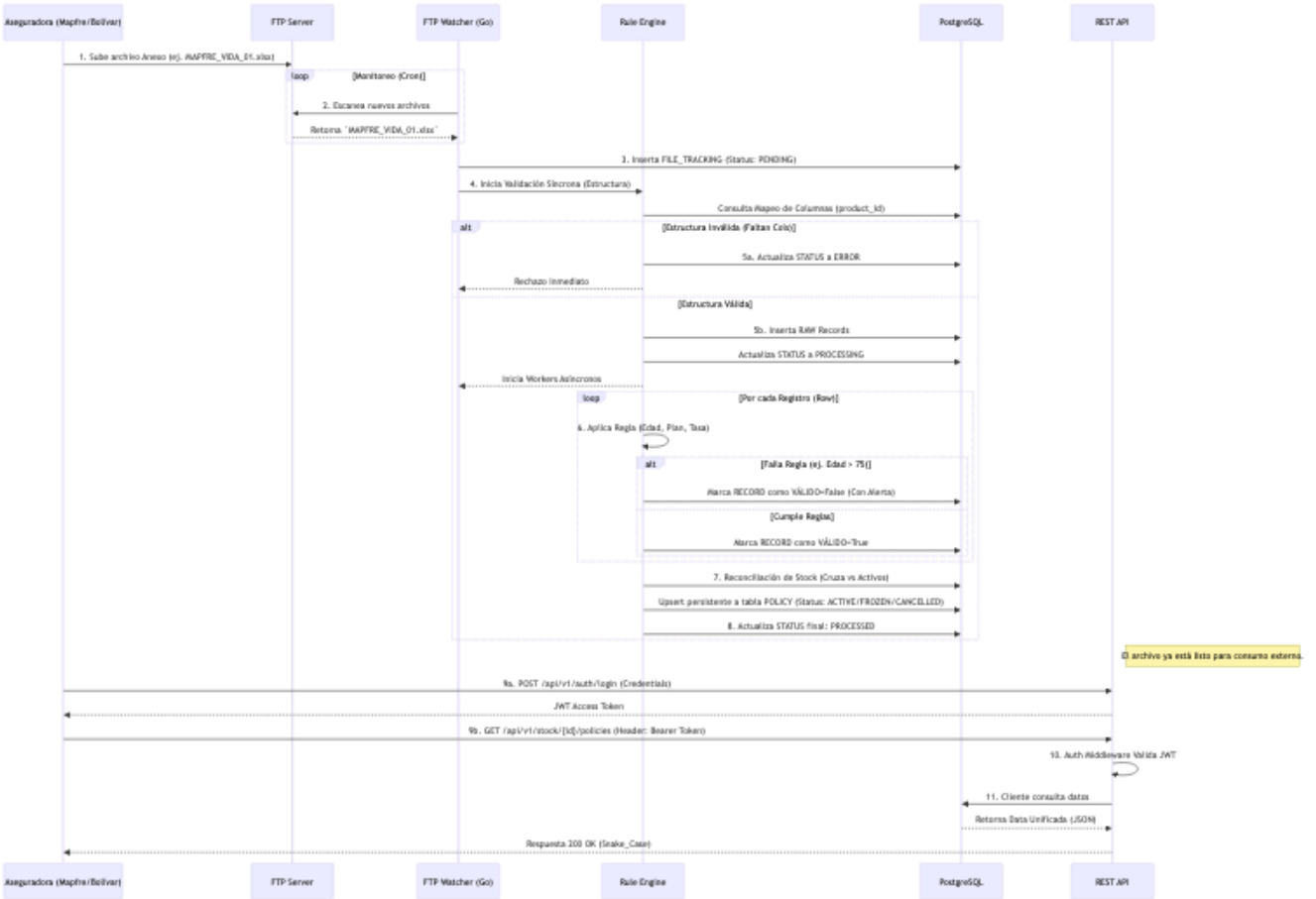
    Broker([Broker Externo])

    W -- Envía RAW file --> P
    P -- Envía Batch --> RE
    RE -- Upsert/Reconcilia --> DB
    Broker -- Peticiones API --> Auth
    Auth -- Rutea req --> API
    API -- Consulta Stock --> DB

```

3. Flujo de Integración y Procesamiento (Secuencia)

El siguiente diagrama detalla la integración End-to-End desde que la aseguradora sube el archivo hasta que queda disponible en la API.



sequenceDiagram

participant ASG as Aseguradora (Mapfre/Bolívar)

participant FTP as FTP Server

participant W as FTP Watcher (Go)

participant RE as Rule Engine

participant DB as PostgreSQL

participant API as REST API

ASG->>FTP: 1. Sube archivo Anexo (ej. MAPFRE_VIDA_01.xlsx)

loop Monitoreo (Cron)

W->>FTP: 2. Escanea nuevos archivos

FTP-->>W: Retorna `MAPFRE\_VIDA\_01.xlsx`

end

W->>DB: 3. Inserta FILE_TRACKING (Status: PENDING)

W->>RE: 4. Inicia Validación Síncrona (Estructura)

RE->>DB: Consulta Mapeo de Columnas (product_id)

alt Estructura Inválida (Faltan Cols)

RE->>DB: 5a. Actualiza STATUS a ERROR

RE-->>W: Rechazo Inmediato

else Estructura Válida

RE->>DB: 5b. Inserta RAW Records

RE->>DB: Actualiza STATUS a PROCESSING

RE-->>W: Inicia Workers Asíncronos

loop Por cada Registro (Row)

RE->>RE: 6. Aplica Regla (Edad, Plan, Tasa)

alt Falla Regla (ej. Edad > 75)

RE->>DB: Marca RECORD como VÁLIDO=False (Con Alerta)

else Cumple Reglas

RE->>DB: Marca RECORD como VÁLIDO=True

end

end

RE->>DB: 7. Reconciliación de Stock (Cruza vs Activos)

RE->>DB: Upsert persistente a tabla POLICY (Status:

ACTIVE/FROZEN/CANCELLED)

RE->>DB: 8. Actualiza STATUS final: PROCESSED

end

note right of API: El archivo ya está listo para consumo externo.

ASG->>API: 9a. POST /api/v1/auth/login (Credentials)

API-->>ASG: JWT Access Token

ASG->>API: 9b. GET /api/v1/stock/{id}/policies (Header: Bearer Token)

API->>API: 10. Auth Middleware Valida JWT

API->>DB: 11. Cliente consulta datos

DB-->>API: Retorna Data Unificada (JSON)
API-->>ASG: Respuesta 200 OK (Snake_Case)

4. Jobs Asíncronos (Background Workers)

El procesamiento del sistema delega las tareas pesadas a colas asíncronas manejadas internamente por **Goroutines** (Go).

Nombre del Job	Disparador	Tarea Principal
FTPWatcherJob	Cron (ej. cada 15 min) o POST /process/scan	Conecta al SFTP de Mapfre/Bolívar, identifica nuevos archivos no procesados y encola la validación síncrona.
FileProcessingJob	Éxito en Validación Estructural	Lee el archivo fila por fila, instancia el Motor de Reglas , re-calcula primas y marca cada RECORD como válido o inválido.
StockReconciliationJob	Fin exitoso del FileProcessingJob	Ejecuta el cruce (DIFF) masivo en PostgreSQL para detectar cuáles pólizas deben marcarse como CANCELLED, FROZEN o ACTIVE.

5. Eventos de Dominio (Event Streaming)

Alineado con las directrices de Zalando, el motor emite **Eventos de Cambio de Datos (Data Change Events)** cuando se altera el stock. Estos eventos pueden ser consumidos por un bus de mensajería (ej. Kafka/RabbitMQ) para que otros microservicios actúen.

Data Change Events:

- **policy_created**: Disparado cuando el motor detecta un document_number nuevo en el mes actual.
- **policy_updated**: Disparado cuando el deudor o asegurado tiene "novedades" (cambio en el valor cobrado o actualización de parentescos).

- **policy_frozen:** Emitido específicamente cuando una prima en 0 dispara la política de congelamiento.
- **policy_cancelled:** Emitido durante el StockReconciliationJob si el asegurado no aparece en el nuevo archivo mensual.

Lifecycle Events (Flujo Operativo):

- **file_validation_failed:** Se emite si el Anexo viene corrupto, notificando al equipo de operaciones para que exija corrección a la aseguradora.
- **file_processing_completed:** Notifica que el lote completo terminó y el API ya expone el stock fresco.

Seguridad y Control de Acceso

Dado que Busk Seguros procesa archivos que contienen información sensible sobre pólizas de asegurados e información financiera y de deudores (como reportes de MAPFRE y Bolívar), todos los accesos programáticos están resguardados por un esquema de seguridad perimetral.

Modelo de Autenticación (JWT Bearer)

El sistema de la API REST implementa un modelo de seguridad basado en tokens temporales, utilizando el estándar **JSON Web Token (JWT)**, alineado con las especificaciones técnicas requeridas.

1. Interceptor / Auth Middleware

Toda petición entrante que busque consultar el "stock" o disparar comandos de control (/process/scan) debe pasar por un Middleware escrito en Go, el cual se asegura de:

- Verificar que el Token posea una firma válida emitida por el backend.
- Validar que el Token no esté expirado.
- (Opcional) Confirmar que el rol del usuario contenga los permisos (Scopes) necesarios para dicha operación.

2. Tabla de Credenciales y Roles (APP_USER)

El almacenamiento interno de usuarios y contraseñas (hasheadas) se realiza mediante la entidad de base de datos dedicada.

Tabla	Campo	Tipo	Notas / Negocio
APP_USER	id	UUID (PK)	Identificador interno
	email	STRING (UK)	Credencial de acceso (Ej. operaciones@busk.com)
	password_hash	STRING	Criptografía irreversible (Bcrypt / Scrypt)
	role	STRING	ADMIN, VIEWER, SYSTEM
	is_active	BOOLEAN	Corta acceso instantáneamente sin borrar registro

	last_login_at	TIMESTAMP	Registro de auditoría.
--	---------------	-----------	------------------------

Flujo de Autorización

1. Intercambio de Credenciales (Login)

El integrador de software o portal web frontal (Frontend) consume el endpoint de inicio de sesión documentado en las [Firmas de API](#).

```
POST /api/v1/auth/login
Content-Type: application/json

{
  "email": "asesor@busk.com",
  "password": "SuperSecretPassword123"
}
```

Si es correcto, la API devuelve el access_token temporal.

2. Consumo de API (Autorización)

El cliente debe proveer dicho access_token inyectándolo en las cabeceras estándar de Autorización (Authorization) en cada petición subsecuente.

```
GET /api/v1/stock/MAPFRE_VIDA_XYZ/policies HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI...
```

Respuestas de Seguridad (RFC 7807)

Alineado con Zalando Guidelines, un error de autenticación se presentará mediante application/problem+json:

```
{  
  "type": "https://docs.buskseguros.com/errors/unauthorized",  
  "title": "Unauthorized Request",  
  "status": 401,  
  "detail": "El token JWT provisto ha expirado o es inválido."  
}
```

Proceso: FTP, Prefijos y Validación

Este documento detalla la fase inicial y crítica del procesamiento de archivos.

1. Recuperación vía FTP

El sistema utiliza un cliente interno que se conecta periódicamente a servidores FTP/SFTP externos para buscar nuevos documentos de seguros.

2. Identificación por Prefijo

Para saber qué reglas aplicar, el sistema analiza el nombre del archivo buscando un **Prefijo Clave**:

Prefijo	Producto Correspondiente	Anexo
MAPFRE_VIDA_	Voluntario Vida	Anexo 1
MAPFRE_CANCER_	AP Cáncer	Anexo 2
MAPFRE_MENORES_	AP Menores	Anexo 3
BOLIVAR_BANCO_	Deudores Banco	Anexo 4
BOLIVAR_ESAL_	Deudores ESAL	Anexo 5

3. Validación de Estructura (Síncrona)

¡CRÍTICO! Antes de cualquier inserción, el sistema valida que el archivo coincida exactamente con la parametrización guardada.

- **Verificación:** Número de columnas, nombres exactos de encabezados y orden.
- **Reporte de Error:** Si hay un cambio estructural, el sistema:
 1. Detiene el proceso inmediatamente.
 2. Registra el error en PROCESSED_FILE.
 3. Notifica al equipo operativo.
- **Éxito:** Si la estructura es correcta, se registra como **PENDIENTE** en la base de datos.

4. Procesamiento Asíncrono

Una vez aceptado el archivo, los registros se validan fila por fila de forma asíncrona:

1. Se ejecutan validaciones por columna (Edad, Plazo, Tasa).
2. Se guardan los resultados individualmente en la tabla RECORD.
3. Al finalizar, el estado del archivo cambia a PROCESADO o ERROR (si hubo fallas en registros).

Firmas de API (Hub Técnico)

Esta sección documenta los endpoints REST disponibles para controlar y monitorear el procesamiento automatizado desde el FTP.

1. Control de Procesamiento

POST /api/v1/process/scan

Dispara manualmente el escaneo del servidor FTP para buscar nuevos archivos.

Descripción: El sistema busca archivos que coincidan con los prefijos configurados (MAPFRE_, BOLIVAR_).

Respuestas:

- 200 OK: Escaneo iniciado exitosamente.
- 503 Service Unavailable: Servidor FTP no disponible.

GET /api/v1/files

Lista todos los archivos detectados en el FTP con su estado actual.

Filtros opcionales: status (PENDIENTE, ERROR, PROCESADO, FROZEN), productId, date.

GET /api/v1/files/:id/diff

Obtiene el reporte de diferencias (Inclusions, Cancellations y Novedades) una vez el proceso asíncrono termina.

Esquema de Retorno:

```
{
  "filename": "MAPFRE_VIDA_202403.xlsx",
  "status": "PROCESSED",
  "inclusions": [ ... ],
  "cancellations": [ ... ],
  "novedades": [ ... ]
}
```

2. Gestión de Stock e Inventario

GET /api/v1/stock/:productId/policies

Consulta el listado de pólizas activas en el stock para un producto específico.

Parámetros de Consulta (Query):

- document_number: Filtrar por número de documento (Cédula/NIT).
- credit_id: Filtrar por ID de crédito (Bolívar).
- full_name: Filtrar por nombre del asegurado (Búsqueda parcial).
- status: Filtrar por estado (ACTIVE, FROZEN, CANCELLED).
- page / limit: Control de paginación.

GET /api/v1/stock/:productId/policies/:id

Obtiene el detalle completo de una póliza o crédito. El esquema es **unificado** para todos los productos, permitiendo que el cliente procese la información de manera genérica.

Esquema Unificado (Base):

Nodo	Descripción
customer_data	Información del titular/asegurado/deudor.
financial_data	Valores monetarios, tasas y primas.
reference_data	Datos específicos del origen (Póliza Mapfre o Crédito Bolívar).
validation_data	Resultados de reglas y alertas.

Ejemplo: Producto MAPFRE (Vida Voluntaria)

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "product_id": "MAPFRE_VIDA",
  "status": "ACTIVE",
  "customer_data": {
    "id_number": "12345678",
    "id_type": "CC",
    "full_name": "JUAN PEREZ",
    "gender": "M",
    "birth_date": "1985-05-15"
  },
  "financial_data": {
    "base_value": 0,
    "premium_value": 8600.00,
    "tax_value": 0,
    "total_value": 8600.00,
    "currency": "COP"
  },
  "reference_data": {
    "reference_number": "108-000456",
    "start_date": "2024-01-01",
    "end_date": "2024-12-31",
    "additional_system_info": {
      "branch": "BOGOTA",
      "plan": "PLAN A"
    }
  },
  "validation_data": {
    "is_valid": true,
    "alerts": [],
    "calculated_age": 38
  }
}
```

Ejemplo: Producto BOLIVAR (Deudores)

```
{
  "id": "660e8400-e29b-41d4-a716-446655441111",
  "product_id": "BOLIVAR_BANCO",
  "status": "ACTIVE",
  "customer_data": {
    "id_number": "900123456",
    "id_type": "NIT",
    "full_name": "EMPRESA ABC SAS",
    "gender": "N/A",
    "birth_date": null
  },
  "financial_data": {
    "base_value": 25000000.00,
    "premium_value": 20825.00,
    "tax_value": 0,
    "total_value": 20825.00,
    "currency": "COP",
    "applied_rate": 0.000833
  },
  "reference_data": {
    "reference_number": "99887766",
    "start_date": "2023-11-20",
    "end_date": null,
    "additional_system_info": {
      "term_months": 60,
      "operation_type": "BT"
    }
  },
  "validation_data": {
    "is_valid": true,
    "alerts": ["MANUAL_REVIEW_REQUIRED"],
    "requires_manual_action": true
  }
}
```

POST /api/v1/stock/reload

Reemplaza la base de stock actual con los datos consolidados del último archivo procesado exitosamente.

GET /api/v1/rules/:productId

Consulta la parametrización de estructura y reglas aplicada para un tipo de archivo específico.

3. Manejo de Errores (RFC 7807)

Todos los errores devueltos por la API seguirán la especificación **Problem JSON (RFC 7807)** requerida por Zalando Guidelines.

```
{
  "type": "https://docs.buskseguros.com/errors/resource-not-found",
  "title": "Stock Not Found",
  "status": 404,
  "detail": "No active policies found for product MAPFRE_VIDA_XYZ.",
  "instance": "/api/v1/stock/MAPFRE_VIDA_XYZ/policies"
}
```

4. Especificación OpenAPI 3.0 (Swagger)

A continuación, el contrato formal de integración para el consumo de políticas validadas y control de archivos.

```
openapi: 3.0.3
info:
  title: Busk Seguros Docs API
  version: 1.0.0
  description: "API for accessing validated insurance policies and controlling
FTP file ingestion."
servers:
  - url: https://api.buskseguros.com/v1
security:
  - bearerAuth: []
paths:
  /auth/login:
    post:
      summary: Authenticate user and get JWT
      operationId: loginUser
      security: [] # No auth required to login
      tags:
        - Authentication
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [email, password]
              properties:
                email:
                  type: string
                  format: email
                password:
                  type: string
                  format: password
      responses:
        '200':
          description: Successful authentication
          content:
            application/json:
              schema:
                type: object
                properties:
                  access_token:
                    type: string
                  expires_in:
                    type: integer
        '401':
          description: Invalid credentials
          content:
            application/problem+json:
              schema:
                $ref: '#/components/schemas/Problem'
```

```
/process/scan:
  post:
    summary: Trigger manual FTP scan
    operationId: triggerScan
    tags:
      - Control
    responses:
      '200':
        description: Scan initiated successfully
      '503':
        description: FTP service unavailable
        content:
          application/problem+json:
            schema:
              $ref: '#/components/schemas/Problem'

/files:
  get:
    summary: List all detected files
    operationId: listFiles
    tags:
      - Monitoring
    parameters:
      - in: query
        name: status
        schema:
          type: string
          enum: [PENDING, PROCESSING, PROCESSED, ERROR]
      - in: query
        name: product_id
        schema:
          type: string
    responses:
      '200':
        description: A list of file tracking records

/files/{id}/diff:
  get:
    summary: Get processing diff for a specific file
    operationId: getFileDiff
    tags:
      - Monitoring
    parameters:
      - in: path
        name: id
        required: true
        schema:
          type: string
          format: uuid
    responses:
```

```
    '200':
      description: Diff report containing inclusions, cancellations, and
novedades
```

```
/stock/reload:
```

```
  post:
    summary: Reload persistent stock from the latest processed file
    operationId: reloadStock
    tags:
      - Stock Management
    responses:
      '200':
        description: Stock reloaded successfully
```

```
/rules/{product_id}:
```

```
  get:
    summary: Get rule configuration for a product
    operationId: getProductRules
    tags:
      - Configuration
    parameters:
      - in: path
        name: product_id
        required: true
        schema:
          type: string
    responses:
      '200':
        description: Rule configuration
```

```
/stock/{product_id}/policies:
```

```
  get:
    summary: Retrieve active policies for a product
    operationId: getStockPolicies
    tags:
      - Stock
    parameters:
      - in: path
        name: product_id
        required: true
        schema:
          type: string
      - in: query
        name: document_number
        schema:
          type: string
      - in: query
        name: credit_id
        schema:
          type: string
      - in: query
```

```

    name: status
    schema:
      type: string
      enum: [ACTIVE, FROZEN, CANCELLED]
  responses:
    '200':
      description: A list of policies
      content:
        application/json:
          schema:
            type: object
            properties:
              items:
                type: array
                items:
                  $ref: '#/components/schemas/Policy'
    '400':
      description: Bad Request
      content:
        application/problem+json:
          schema:
            $ref: '#/components/schemas/Problem'
    '404':
      description: Not Found
      content:
        application/problem+json:
          schema:
            $ref: '#/components/schemas/Problem'

components:
  schemas:
    Policy:
      type: object
      properties:
        id:
          type: string
          format: uuid
        product_id:
          type: string
        status:
          type: string
        customer_data:
          type: object
        financial_data:
          type: object
        reference_data:
          type: object
        validation_data:
          type: object
    Problem:
      type: object

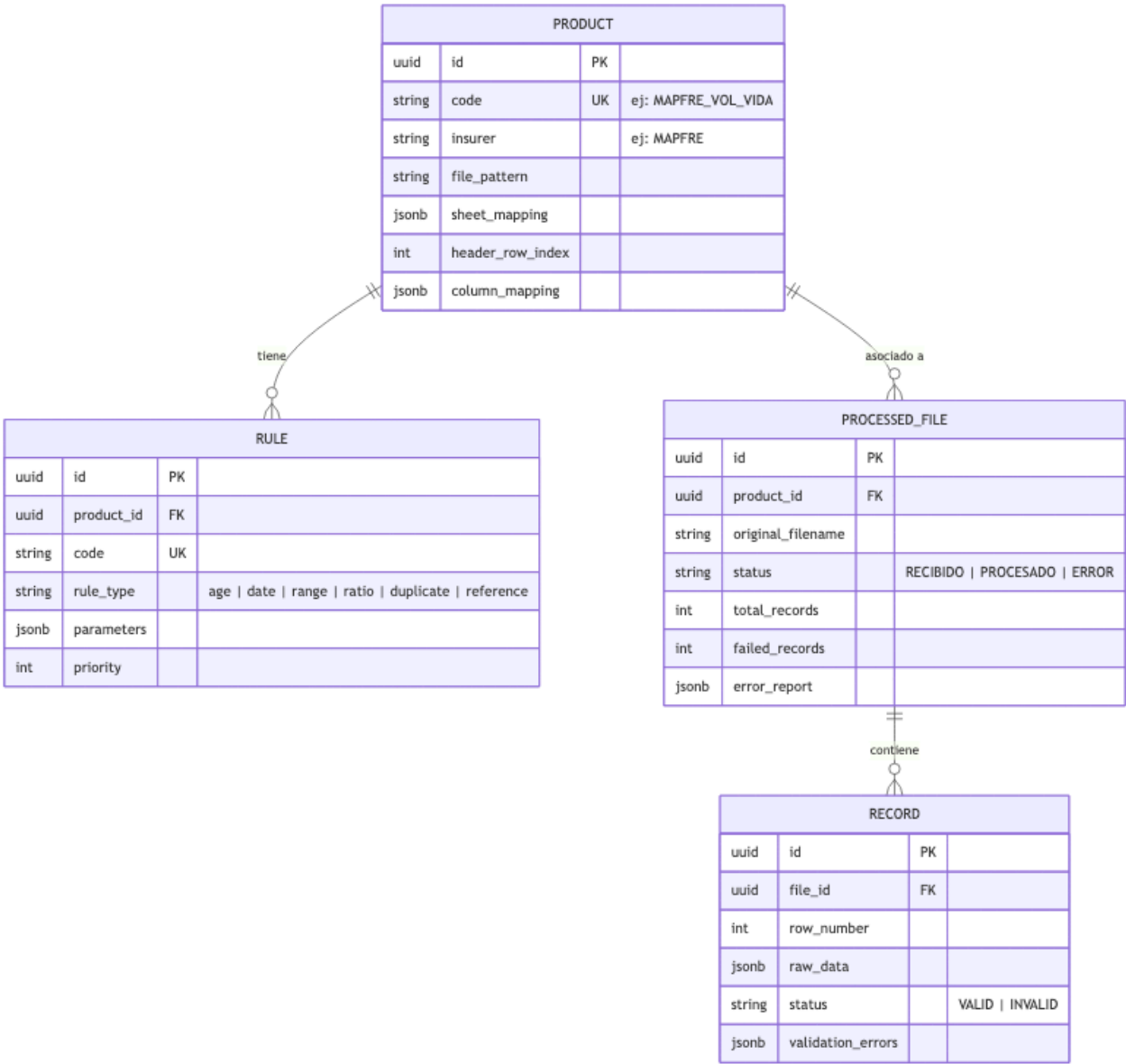
```

```
properties:
  type:
    type: string
    format: uri
  title:
    type: string
  status:
    type: integer
  detail:
    type: string
  instance:
    type: string
    format: uri
securitySchemes:
  bearerAuth:
    type: http
    scheme: bearer
    bearerFormat: JWT
    description: "Enter your Bearer token in the format **Bearer
<token>**"
```

Esquema de Base de Datos Completo

El sistema utiliza PostgreSQL para almacenar la trazabilidad del proceso y el stock histórico.

Modelo de Datos Visual



Diccionario de Datos

Tabla	Columna	Tipo	Lógica de Negocio
PRODUCT	id	UUID (PK)	Identificador único

	code	STRING (UK)	ID interno (ej. MAPFRE_VIDA)
	insurer	STRING	Aseguradora: MAPFRE o BOLIVAR
	column_mapping	JSONB	Mapeo de columnas Excel a campos DB
POLICY (Stock Activo)	id	UUID (PK)	
	product_id	UUID (FK)	
	document_type	STRING	CC, NIT, TI, etc.
	document_number	STRING	Indexado (Llave Mapfre)
	credit_id	STRING	Indexado (Llave Bolívar)
	full_name	STRING	Indexado (Búsqueda rápida)
	birth_date	DATE	Para validación de edad
	gender	STRING	M, F, N/A
	status	STRING	Indexado (ACTIVE, FROZEN, etc.)
	premium_value	NUMERIC	Valor para cálculos inmediatos
	metadata	JSONB	Otros campos específicos (Beneficiarios, etc.)
	updated_at	TIMESTAMP	
RULE	id	UUID (PK)	
	product_id	UUID (FK)	Producto padre
	rule_type	STRING	edad, plan, tasa, duplicados

	parameters	JSONB	Ej. {"max_age": 75, "min_age": 18}
--	------------	-------	------------------------------------

Seguridad y Control de Acceso (Usuarios)

Para garantizar la confidencialidad de la información (pólizas), el API está resguardada por autenticación JWT.

Tabla	Campo	Tipo	Notas / Negocio
APP_USER	id	UUID (PK)	Identificador interno
	email	STRING (UK)	Credencial de acceso
	password_hash	STRING	Bcrypt / Scrypt
	role	STRING	ADMIN, VIEWER, SYSTEM
	is_active	BOOLEAN	Control de acceso rápido
	last_login_at	TIMESTAMP	Auditoría

| PROCESSED_FILE | id | UUID (PK) | || | status | STRING |
RECIBIDO, PENDIENTE, PROCESANDO, LISTO, ERROR || |
error_report | JSONB | Reporte detallado de errores de
validación || RECORD | id | UUID (PK) | || | file_id | UUID
(FK) | || | raw_data | JSONB | Contenido completo de la fila
original || | status | STRING | VÁLIDO o INVÁLIDO |

Recomendaciones de Escalabilidad (Millones de Registros)

Para manejar millones de registros eficientemente en PostgreSQL sin migrar a una NoSQL, se seguirán estas estrategias:

1. Esquema Híbrido (Propuesto)

En lugar de un JSON masivo, extraemos los campos que se comparten entre todos los productos (Nombre, ID, Estado, Valor) a **columnas físicas**.

- **Ventaja:** Las búsquedas por Nombre o Cédula no tocan el JSON, optimizando el uso de memoria y CPU.
- **Flexibilidad:** El resto de campos (ej. "Parentesco del beneficiario" en Mapfre o "Plazo" en Bolívar) se queda en el JSON.

2. Índices B-Tree vs GIN

- Usaremos **B-Tree** convencionales para las columnas externas.
- Usaremos **GIN** solo en el campo `metadata` para búsquedas sobre campos que no logramos prever como compartidos.

2. Particionamiento de Tablas

Para evitar que una sola tabla crezca indefinidamente, se recomienda **Particionamiento por Lista** usando el `product_id`.

- Cada producto (Mapfre Vida, Bolivar Banco, etc.) tendrá su propia partición física en disco, mejorando drásticamente el rendimiento de las consultas y el mantenimiento.

3. Compresión TOAST

PostgreSQL gestiona grandes bloques de datos (como JSONs extensos) mediante **TOAST**, almacenándolos fuera de la tabla principal y comprimiéndolos automáticamente, lo que mantiene los índices pequeños y rápidos.

[!NOTE] Con estas técnicas, PostgreSQL puede manejar cientos de millones de registros manteniendo la integridad relacional (ACID), algo que se pierde o se vuelve complejo en una NoSQL pura como MongoDB.

Catálogo de Productos y Estructuras (Anexos)

El sistema soporta 5 productos principales, cada uno basado en la estructura de los archivos anexos proporcionados.

1. MAPFRE - Vida Voluntaria (Anexo 1)

- **Prefijo Sugerido:** MAPFRE_VIDA_
- **Columnas Clave:**
 - NUM DOCUM: Llave primaria.
 - FECHA NAC: Para validación de edad (18-75 años).
 - PRIMA MENSUAL: Valores válidos (\$8,600 o \$17,100).
- **Características:** Archivo extenso (>100 columnas) con datos de beneficiarios y tomador.

2. MAPFRE - AP Cáncer (Anexo 2)

- **Prefijo Sugerido:** MAPFRE_CANCER_
- **Columnas Clave:**
 - NUM DOCUM: Llave primaria.
 - FECHA NAC: Validación de edad (18-70 años).
 - VALOR PRIMA: \$8,500 o \$13,000.

3. MAPFRE - AP Menores (Anexo 3)

- **Prefijo Sugerido:** MAPFRE_MENORES_
- **Columnas Clave:**
 - NUM DOCUM: Llave primaria.
 - PRIMA: \$7,800, \$7,410, \$10,600 o \$10,070.

4. BOLIVAR - Deudores Banco (Anexo 4)

- **Prefijo Sugerido:** BOLIVAR_BANCO_
- **Columnas Clave:**
 - NUMERO DE CREDITO: Llave primaria.
 - DEUDA INICIAL: Base para cálculo de prima.
 - PRIMA MENSUAL: Validada mediante $DEUDA * TASA$.
 - PLAZO: Validado contra fechas de adjudicación y vencimiento.

5. BOLIVAR - Deudores ESAL (Anexo 5)

- **Prefijo Sugerido:** BOLIVAR_ESAL_
- **Columnas Clave:**
 - Similar a Deudores Banco pero con tasa y reglas específicas para Entidades Sin Ánimo de Lucro (ESAL).
 - Validación de créditos > \$20M (Requiere revisión manual).

Lógica de Negocio y Flujo de Datos

1. Origen y Identificación

El sistema no solo monitorea carpetas locales, sino que se conecta a un **servidor FTP** para descargar archivos.

Identificación por Prefijo:

Cada archivo es identificado mediante un **Prefijo o Nombre Clave**. Esto permite al sistema saber instantáneamente qué parametrización de producto aplicar. Ejemplo:

MAPFRE_VIDA_202403.xlsx -> Producto: MAPFRE Voluntario Vida.

2. Validación de Estructura (Síncrona)

Antes de procesar cualquier registro, el sistema verifica que la **estructura del archivo** sea idéntica a la parametrizada (nombres de columnas, orden y tipos).

- **Si cambia la estructura:** Se detiene el proceso y se reporta el error de inmediato sin cargar datos.
 - **Si la estructura es válida:** El archivo se registra en la base de datos con estado **"PENDIENTE"** para su procesamiento posterior.
-

3. Procesamiento Asíncrono de Registros

Una vez que el archivo es aceptado estructuralmente, un proceso asíncrono recorre cada registro para ejecutar las validaciones de columna (Edad, Plan, Tasa, etc.).

Flujo de Registro:

1. **Validación de Columna:** Se ejecutan las reglas individuales.
2. **Persistencia:** Se guarda el resultado de cada registro individualmente.
3. **Cruce de Stock:** Se determinan Inclusiones, Cancelaciones y Novedades.

Reglas Específicas del Diagrama Base (PDF Original):

El Motor Asíncrono debe programar e instanciar obligatoriamente las siguientes reglas funcionales extraídas del modelo de flujo:

A. Reglas Comunes y de Vida (Mapfre)

- **Regla de Edad:** Se valida contra el parámetro del producto (ej. *18 a 75 años* para Vida Voluntario, *18 a 70 años* para AP Menores/Cáncer). Un deudor que incumpla se marca con Novedad/Error.
- **Regla de Plan (Tarifario):** La PRIMA (ej. \$8,600 o \$17,100) debe estar en el diccionario estricto del producto.
- **Control de Siniestros:** Cruce de la Cédula (NUM DOCUM) contra la tabla o servicio externo de fallecimientos reportados. Genera marca de alerta inmediata.

B. Reglas de Crédito (Bolívar)

- **Control de Tasa:** $DEUDA\ INICIAL \times \% \text{ Tasa}$ debe ser exactamente igual a la PRIMA MENSUAL reportada.
- **Montos Superiores a 20 Millones:** Si $DEUDA\ INICIAL > \$20,000,000$, el sistema aprueba la estructura pero levanta bandera MANUAL_REVIEW_REQUIRED (No se emite automáticamente).
- **Control de Duplicidad:** Búsqueda en Stock Histórico e Interno para asegurar que la Operación (OP BT) no se ha cruzado doblemente en el mismo mes.

2. Prioridades del Motor de Reglas

Prioridad	Nivel	Acción ante Falla
1	Estructura	Rechazar archivo completo
2	Datos Críticos	Marcar registro como INVÁLIDO, rechazar archivo
3	Política de Negocio	Marcar como Novedad, requiere revisión manual
4	Metadatos	Advertencia de baja prioridad

4. Reconciliación y Gestión de Vigencia

Para determinar qué pólizas siguen vigentes, el sistema realiza una **Reconciliación de Stock** al finalizar el procesamiento asíncrono.

Lógica de Desactivación:

1. **Identificar Vigencia:** Se seleccionan todas las pólizas en la tabla POLICY que pertenecen al product_id del archivo actual.

2. **Cruce (Diff):** El sistema busca cuáles de esas pólizas **NO** están presentes en el nuevo archivo cargado.
 3. **Desactivación Automática:** Aquellas pólizas ausentes en el nuevo archivo se marcan automáticamente como CANCELLED o INACTIVE.
 4. **Actualización:** Las pólizas que sí están presentes se actualizan con la nueva información del archivo (Novedades).
-

5. Política de Congelamiento

Una regla especial aplicada durante la validación asíncrona es el **Congelamiento de Póliza**.

- **Condición:** Si el valor de la prima ($\text{premium_value} / \text{prima}$) llega en **0** en el archivo para ciertos productos.
- **Acción:** La póliza NO se cancela ni se marca como error. En su lugar, el sistema la marca con el estado **FROZEN (Congelada)**.
- **Efecto:** La póliza sigue marcada como "Activa" bajo esta política especial, permitiendo que la cobertura se mantenga o se suspenda según la lógica interna, pero sin perder la persistencia en el stock.